

Final Project

The goal of this project is to evaluate and understand the contribution of different aspects in the design of relational deep learning models, based on message passing GNNs. To that end, you will design and conduct an *ablation* study: a series of experiments disabling various aspects of message passing GNNs and measuring their impact.

As a main requirement, you will submit a report, similar to a research paper. The report should clearly present the work you did, and it can also be shared externally, e.g., with job interviewers.

For each aspect, the project report should include:

1. The design of your ablation experiment: how do you plan to evaluate the studied aspect. This should include: your choice of datasets and prediction tasks, any specific data preparation you perform, evaluation measures, your choice of models out of the recommended ones (if applicable), your specific instantiations of them, and any other information relevant to your design.

In your design, consider and explain how you plan to make the comparison between different variations (ablated or not) fair, meaning - how you isolate the change only to the specific aspect (for example, use the same number of layers and learned parameters, the same input features, as much as possible).
2. Experiment results, including suitable visualizations (figures, tables, etc.).
3. Discussion on the results, comparison against your initial expectations, and explanations of observed phenomena.

Datasets and tasks: In your experiments, you can use any datasets from [rel-bench](#), and any **entity classification** tasks within these datasets. As performance measures in the downstream tasks, use at least ROC AUC, precision, recall, and AUPRC (area under the precision-recall curve).

A note on scale: to handle large scale datasets you are encouraged to use any of the mini batching techniques we covered (e.g. neighbor sampling methods or cluster-GCN for homogeneous settings). In extreme cases, consider sampling a subset of the dataset. If you do, document the process in your report and explain how you performed the sample to preserve the properties of the learning tasks.

Useful tools: Here are some useful tools you can use, but you are not limited to them.

- `relbench.datasets.get_dataset` and `relbench.tasks.get_task` - fetching the datasets and tasks from relbench.
- `torch_geometric.data.Data` - represents a feature node matrix and an edge_index.
- `torch_geometric.data.HeteroData` - allows storing different node types and edge types in separate dictionaries.

- `torch_geometric.loader.NeighborLoader` - mini batching by sampling a fixed number of neighbors per layer. Using the `time_attr` argument, it also enables to enforce temporal constraints. Note that in the heterogeneous setting, the number of neighbors is specified per layer **and edge type**.
- `torch_geometric.nn.to_hetero` - a PyG wrapper to automatically convert a homogeneous GNN model into heterogeneous, using automatic duplication of components (layers, parameters and message passing logic) to support different types of nodes and edges.

Your submission should include:

- A comprehensive report of your work, describing all of the above, including an introduction and summary (see below).
- All code you used in your work, as runnable python files or ipynb notebooks.
- An ipynb notebook containing all results (figures, tables, etc.) - exported to html format.

Evaluating Aspects of Relational Deep Learning

Aspect 1: Message Directionality

We consider the graph induced from a relational DB where nodes represent tuples, connected by PK-FK edges. When designing a message passing GNN, it is possible to consider several modes of directionality. The choice of directionality mode directly affects the information flow in the GNN. As shown in module 5, modes of directionality can be:

- MPNN-U - for every edge (u,v), messages are passed from u to v and from v to u.
- MPNN-D - for every edge (u->v), messages are passed only from u to v.
- Dir-GNN - messages are passed on both directions of the edge, but a separate message aggregation is done for each.

Design an experiment to compare how the three modes of directionality affect downstream performance. Use at least two datasets. Select at least one model out of: GraphSAGE, Graph Attention (GAT), RGCN.

Aspect 2: Heterogeneity

We are interested in evaluating the homogeneous versus heterogeneous graph approaches. Compare between the two variations:

- Homogeneous - nodes and edges have no labels. Node representations must be of the same form (even if they are from different tables). To achieve that, at the beginning of your architecture use an encoding linear layer (different instance for each table). This layer gets a vector representing the encoding of a tuple, and outputs a vector of a fixed size. You can use `torch.nn.ModuleDict()` to store the linear encoder for each table (e.g., `torch.nn.Linear(table_in_dim, fixed_hidden_dim)`).
- Heterogeneous - nodes and edges may have labels (representing their types). Different node types may have different forms of initial representation.

Design an experiment to test if the heterogeneous approach improves performance over prediction tasks. Use at least 2 models out of: GraphSage, GAT (compare to the heterogeneous versions of them using `to_hetero`), HGT (disable the heterogeneity by building `HeteroData` such that all nodes have the same type, and all edges have same type). Use at least two datasets.

Aspect 3: Node Features

Before training an MPGNN, each node has an initial representation. Focus on the **heterogeneous** setting, and design an experiment to test how various strategies of the initial node representation affect (1) downstream performance, (2) model complexity (e.g., learned parameters) and (3) usability (how easy it is to implement).

Representation strategies:

- Id encoding (no features baseline) - ignore the tuple features, assign an ID to each node.
- Column-wise encoding - encode each cell (based on the column type) into a numeric representation, and create a vector of the encoded values. Note that such encoders exist already - no need to implement this yourselves.
- LLM encoding - transform each tuple to a string based on the features and values: "A1=v1, A2=v2,..." and embed the string using a language model such as [sentence-transformers](#).

Note: LLM encoding may be heavy in computations. Therefore, you may select a reasonable size sample of the dataset for this experiment. If you do, explain how you performed the sample to preserve the properties of the learning tasks, and make sure to use the same sample for all 3 variations. Use at least 2 datasets. Use the HGT model.

Aspect 4: Limitations of Deeper Models

When increasing the depth of MPNNs (adding more layers), the model is prone to oversmoothing, where node representations become similar and lose their specifications.

Design an experiment to test the sensitivity of a model to oversmoothing. Select one of the following: GCN, GAT, or GraphSAGE (with mean aggregation). The experiment should inspect oversmoothing both in terms of similarity measures (like those shown in tutorial 7), and the effect on downstream performance.

As a follow-up experiment, test the efficacy of one of the following mitigation strategies: (1) skip connections or (2) drop edge, on the model you chose.

Aspect 5: Foundation Models (Dry Question)

Consider one heterogeneous model from your experiments. What changes are required so that the model could be used as a foundation model? In other words, what are the required changes in order to pretrain the model on one dataset and use it on another dataset with a different schema, such that the pretraining can improve the performance?

How would you design an experiment to test the effectiveness of the model?

Summary

- a. Overview: give a high-level overview of the experiments you designed in each aspect, what you expected to find, the main results, and conclusions from each of them.
- b. Future work: what are the limitations of your project, and what could be good directions for followup research?
- c. AI usage: add a paragraph to your report, describing in detail **where** you used AI tools LLMs (such as ChatGPT, Gemini, etc.). Explain your **critical validation process**: how did you catch hallucinations or optimize the AI's suggestions for the specific tasks? Additionally, estimate the **percentage** of the total workflow in the assignment that was assisted by AI tools. **You can use AI as much as you want, as long as you understand what it did, validate that it was correct, and report it.**